

B 3.2.5 Design patterns in OOP Python (HL Only)



B3.2.5 Explain commonly used design patterns in OOP. (HL Only)

This section explains common OOP design patterns, including singleton, factory, and observer. It explores how these patterns are applied to solve recurring programming problems.

What are Design Patterns?

Design patterns in software engineering are time-tested, reusable solutions to frequently encountered problems in software design. They are not ready-made code snippets but rather conceptual descriptions of how to organise objects and classes to address a specific design issue. They are established best practices or templates that guide your architectural choices.

Employing design patterns in your Python projects yields several benefits:

- **Reusability:** They offer proven solutions that can be adapted across different projects.
- **Maintainability:** Code adhering to recognized patterns is generally easier for other developers (and your future self!) to understand and modify.
- **Scalability:** Patterns can aid in designing systems capable of growing and adapting to evolving requirements.
- **Communication:** A shared vocabulary of design patterns improves communication and understanding within development teams.

The Singleton Pattern

Definition: The Singleton pattern ensures that a class has only one instance and provides a global access point to that instance. This is particularly useful when you need exactly one object to coordinate actions across your Python application.

Use Cases:

- **Managing a database connection:** Typically, you want a single connection pool to manage database resources efficiently.

- Global configuration settings: A single object can hold all the application's configuration parameters.
- Logging: A single logger instance ensures all log messages are directed to the same place.

Python Implementation:

```
class Singleton:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if not cls._instance:
            cls._instance = super(Singleton, cls).__new__(cls, *args, **kwargs)
            print("Singleton instance created.")
        return cls._instance
    def do_something(self):
        print("Singleton is doing something!")

# Example usage
instance1 = Singleton()
instance2 = Singleton()
# Both variables point to the same object
print(instance1 is instance2) # Output: True
instance1.do_something() # Output: Singleton is doing something!
instance2.do_something() # Output: Singleton is doing something!
```

Explanation:

- We define a class `Singleton` with a class-level attribute `_instance` initialized to `None`.
- The `__new__` method is a special method called before `__init__` when creating an object.
- Inside `__new__`, we check if `_instance` is `None`. If it is, we create a new instance of the class using the `super()` call and store it in `_instance`.
- For subsequent attempts to create an instance, the existing `_instance` is returned.
- The example shows that creating multiple "instances" of `Singleton` results in the same object.

The Factory Pattern

Definition: The Factory pattern defines an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created. It essentially moves the object creation logic into a separate factory function or class.

Use Cases:

- Creating different types of shapes (e.g., Circle, Square, Triangle) based on user input.

- Loading different file formats (e.g., PDF, DOC, TXT) without the client needing to know the specific class to instantiate for each format.
- Abstracting the complex initialization process of objects.

Python Implementation (Simple Factory):

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def draw(self):
        pass
class Circle(Shape):
    def draw(self):
        print("Drawing a Circle.")
class Square(Shape):
    def draw(self):
        print("Drawing a Square.")
class ShapeFactory:
    def get_shape(self, shape_type):
        if shape_type is None:
            return None
        if shape_type.lower() == "circle":
            return Circle()
        elif shape_type.lower() == "square":
            return Square()
        return None
# Example usage
factory = ShapeFactory()
circle = factory.get_shape("CIRCLE")
if circle:
    circle.draw() # Output: Drawing a Circle.
square = factory.get_shape("square")
if square:
    square.draw() # Output: Drawing a Square.
unknown = factory.get_shape("rectangle")
if unknown:
    unknown.draw()
else:
    print("Unknown shape type.") # Output: Unknown shape type.
```

Explanation:

- We define an abstract base class **Shape** using Python's **abc** module to enforce an interface.
- **Circle** and **Square** are concrete implementations of the **Shape** interface.
- The **ShapeFactory** class has a **get_shape** method that takes the desired shape type as a string and returns the corresponding **Shape** object. The client code uses the factory to create **Shape**

objects without directly instantiating `Circle` or `Square`. This provides flexibility and reduces dependencies.

The Observer Pattern

Definition: The Observer pattern establishes a one-to-many dependency between objects. When the state of one object (the subject) changes, all its dependent objects (the observers) are automatically notified and updated.

Use Cases:

- Event handling in GUI frameworks: When a user interacts with a UI element, registered observers (like event handlers) are notified.
- Publish-subscribe messaging systems: Publishers broadcast messages, and subscribers receive messages they are interested in.
- Implementing the Model-View-Controller (MVC) architectural pattern, where views are updated when the model changes.

Python Implementation:

```

class Observer:
    def update(self, message):
        pass
class Subject:
    def __init__(self):
        self._observers = []
        self._message = None
    def attach(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)
    def detach(self, observer):
        if observer in self._observers:
            self._observers.remove(observer)
    def notify(self):
        for observer in self._observers:
            observer.update(self._message)
    def set_message(self, message):
        self._message = message
        self.notify()
class ConcreteObserver(Observer):
    def __init__(self, name):
        self.name = name
    def update(self, message):
        print(f"{self.name} received message: {message}")
# Example usage
subject = Subject()
observer1 = ConcreteObserver("Observer 1")
observer2 = ConcreteObserver("Observer 2")
subject.attach(observer1)
subject.attach(observer2)
subject.set_message("New article published!")
# Output:
# Observer 1 received message: New article published!
# Observer 2 received message: New article published!
subject.detach(observer1)
subject.set_message("Breaking news!")
# Output:
# Observer 2 received message: Breaking news!

```

Explanation:

- The **Observer** class defines the **update** method that concrete observers will implement.
- The **Subject** class maintains a list of observers and provides methods to attach, detach, and notify them. The **set_message** method updates the subject's state and triggers the notification.
- **ConcreteObserver** is a specific observer that prints the received message.

- The example demonstrates how observers register with the subject and receive updates when the subject's message changes.

Application of Design Patterns in Solving Recurring Programming Challenges

The design patterns we've explored are invaluable for addressing common challenges in Python development.

- **Singleton:** Effectively manages resources and ensures that only one instance of a crucial class exists, preventing potential conflicts and ensuring consistent behaviour for globally accessed components (e.g., configuration, logging).
- **Factory:** Decouples object creation from the client code, providing a flexible way to instantiate different concrete classes based on specific requirements. This makes the system more adaptable to changes and extensions without modifying the client code directly.
- **Observer:** This function facilitates the creation of loosely coupled systems in which changes in one object automatically propagate to its dependents. This is essential for building reactive and event-driven applications, ensuring different system parts stay synchronised without tight interdependencies.

By mastering these and other design patterns, you can write cleaner, more flexible, and more maintainable Python code, leading to better solutions for recurring programming problems. Remember that the key is to choose the appropriate pattern based on the specific problem you are trying to solve and the overall design goals of your application.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.
